ECSE 429 – Part A

Olivia Cardillo 260253654

Yanick Wiaderek 261183242

October 20th, 2026

Table of Contents

Summary of Deliverables

Findings of Exploratory Testing

1) Todos
   a) Session 1: Todos API - JSON with XML check
2) Projects
   a) Session 1: JSON Exploratory Testing
   b) Supplementary XML Verification

Structure of Unit Test Suite

Source Code Repository

Findings of Unit Test Suite Execution

Use of AI

Summary of Deliverables

This section summarizes the materials prepared for the Projects API portion of Part A.

- Exploratory testing notes from October 12, 2026, covering JSON requests and later supplementary XML verification using Terminal and curl.
- Projects capability demonstration scripts for creating, retrieving, updating, deleting and filtering projects, along with relationship and error-case requests.
- Four bug summary documents (BUG-01 to BUG-04), describing the unexpected behavior of relationship endpoints for nonexistent projects.
- JavaScript automated tests organized into JSON and XML modules, including 43 regular Projects API tests, four observed-behavior bug tests and four proposed expected-behavior checks.
- A test demonstration video showing the 47 passing tests, the four separate expected-behavior failures and a randomized execution of the passing suite.

The exploratory testing focused on identifying documented and undocumented capabilities, normal operations and potential areas of instability. The automated suite was then used to verify these behaviors more consistently and check for unintended changes to the API data.

This section summarizes the materials prepared for the Todos API portion of Part A.

- Exploratory testing notes from October 17, 2026, covering the main Todos operations in JSON, relationships with projects and categories, and a short XML check at the end, all done with curl in the VS Code terminal.
- A Todos capability script that demonstrates creating, retrieving, updating, deleting and filtering todos, along with validation, relationship, XML and undocumented requests.
- Six bug summary documents (BUG-05 to BUG-10), describing the PUT update behavior, the relationship endpoints for nonexistent todos, one-sided todo–category links, XML relationship creation and IDs used up by rejected requests.
- JavaScript automated tests added to the existing JSON and XML modules, including 61 regular Todos API tests, six observed-behavior bug tests and six proposed expected-behavior checks.
- A test demonstration video showing the combined 114 passing tests, the six separate Todos expected-behavior failures and a randomized execution of the passing suite.

I used the session to see how the todo endpoints actually behaved compared to the docs, and wrote the tests afterwards to check those results again and make sure the tests themselves weren’t changing the data.

Findings of Exploratory Testing

1) Todos

The Todos API was tested locally on macOS using runTodoManagerRestAPI-1.5.5.jar at http://localhost:4567. Like Olivia, I mainly used curl, running it from the VS Code terminal to look at status codes, response bodies and headers. Before starting, I restarted the API so the session would begin from the default data. For typical data I used a clothing shopping list. The work is summarized below in the same session-based format used in the sample report.

a) Session 1: Todos API — JSON with XML check (October 17, 2026)
The session lasted 42 minutes and was mostly done in JSON, with a short XML check at the end. I started with the basic operations on the default todos. GET and HEAD returned HTTP 200 for existing todos and HTTP 404 for a todo that did not exist, and creating a todo returned HTTP 201 with an ID assigned by the server. One thing I noticed right away was that doneStatus came back as the string “false” even though I had sent a boolean.
That became more relevant once I started testing the validation rules. Sending doneStatus as a string was rejected with HTTP 400, which means a client that sends back a todo exactly as it received it would get an error. The other rules matched the documentation: a missing or empty title, an unknown field and a client-chosen ID were all rejected with HTTP 400. While doing this I noticed that the IDs were skipping numbers. A todo created after three rejected requests got ID 7 instead of 4, and creating another one right after with no failures in between got the next number. After the session I tested each type of invalid request separately and found that requests failing field validation used up an ID, while requests with a client-chosen ID or malformed JSON did not (BUG-10).
Updating a todo with POST /todos/:id worked as a partial update, and only the field I sent was changed. PUT /todos/:id is described with the same wording in the documentation, but sending only a new title reset doneStatus to false and erased the description, and a GET confirmed this was saved (BUG-05). PUT also required a title every time, so it behaves more like a full replacement than an amendment. Deleting a todo returned HTTP 200, and both the following GET and a second DELETE returned HTTP 404. Filtering by title and by doneStatus returned the expected todos, although unknown query parameters were simply ignored even though unknown fields are rejected when creating a todo.
Next I looked at relationships, mostly to see whether the problem Olivia found on the projects side also happens with todos. It does. GET /todos/999/tasksof returned HTTP 200 with projects that belong to other todos, with “Office Work” listed twice (BUG-06), and GET /todos/999/categories did the same with categories (BUG-07), while GET /todos/999 correctly returned HTTP 404. Linking a real todo to Project 1 and Category 1 both returned HTTP 201. The todo showed up in the project’s task list, but GET /categories/1/todos stayed empty, even for the default todo that is linked to that category (BUG-08). After the session I found that this also happens in the other direction. Deleting a todo linked to two projects removed it from their task lists without deleting the projects.

I also came across two undocumented capabilities. OPTIONS returned the allowed methods in the Allow header, and sending a title instead of an ID to /todos/:id/tasksof created a new project and linked it in one request. PATCH on a todo and DELETE on the whole collection returned HTTP 405.
For the XML check, creating a todo from an XML body worked without problems. Creating a relationship did not: linking a todo to a project with an XML body returned HTTP 404 “Could not find thing matching value for id”, and the same happened with categories, while the identical requests in JSON returned HTTP 201 (BUG-09). A GET afterwards showed nothing had been partially saved.
A few other responses were inconsistent, but I recorded them as issues rather than bugs. Malformed JSON was rejected with HTTP 400 but used a text/html Content-Type and showed the name of an internal Gson exception, a request with a trailing slash returned an HTML 404 with no message, and the API used three different messages for a nonexistent todo depending on the method.
In summary, six potential problems were recorded:
BUG-05: PUT /todos/:id resets fields that are not included in the body.
BUG-06: GET /todos/:id/tasksof for a nonexistent todo returns HTTP 200 and projects belonging to other todos.
BUG-07: GET /todos/:id/categories for a nonexistent todo returns HTTP 200 and categories belonging to other todos.
BUG-08: Todo-category links only appear on the side they were created from.
BUG-09: Relationships cannot be created with an XML body, while the same JSON request works.
BUG-10: Todo creations rejected for invalid field values still use up an ID.
Each bug was reproduced after the session and documented in an individual bug summary with reproduction steps and potential impact. Since BUG-06 and BUG-07 match BUG-01 to BUG-04, the relationship endpoints probably do not check whether the parent object exists. As with the projects bugs, the expected HTTP 404 still needs to be checked against Swagger. BUG-10 also depends on whether IDs are meant to be sequential, which the documentation does not say.

2) Projects

The Projects API was tested locally using runTodoManagerRestAPI-1.5.5.jar at http://localhost:4567 on macOS. I primarily used curl to send HTTP requests and inspect status codes, response bodies and headers. The work is summarized below in the same session-based format used in the sample report.

a) Session 1: Projects API — JSON (October 12, 2026)

This session focused on the main Projects endpoints using JSON. I tested GET, HEAD, POST, PUT and DELETE requests, title filtering, invalid IDs and associations with existing Todos and Categories. Most core operations behaved as expected. Creating a project returned HTTP 201 with an automatically assigned ID, while retrieving a project or a project collection returned

HTTP 200. GET and HEAD requests for a nonexistent individual project returned HTTP 404, and deleting an existing project was confirmed by a subsequent GET returning 404.
I also compared the two update methods. POST /projects/:id allowed a partial update: changing the description did not remove the existing title. PUT /projects/:id behaved differently by resetting an omitted description to an empty string. A GET request after each update confirmed that these changes were saved. Title filtering correctly returned a matching project or an empty array when no match existed.
The API accepted the creation of a project without a title, saving an empty title. An intentionally malformed JSON request was rejected with HTTP 400. However, the malformed-request response used a text/html Content-Type even though it contained structured error information. I also observed that the active and completed values appeared as strings in JSON and that different methods used different wording for errors involving invalid IDs. These observations require comparison with the documented schema before they can be classified as defects.
I tested project–Todo associations by temporarily linking Todo 1 to another project. The relationship appeared in the project's task list and in the Todo data. After removing the relationship, Todo 1 still existed and retained its original association with Project 1. Project–Category associations were also created and removed with temporary project data.
Four potential problems were recorded when a nonexistent project ID was used with its relationship endpoints:
BUG-01: GET /projects/99999/tasks returned HTTP 200 and tasks 1 and 2 belonging to Project 1, rather than indicating that Project 99999 did not exist.
BUG-02: HEAD /projects/99999/tasks returned HTTP 200, the same status seen for an existing project.
BUG-03: GET /projects/99999/categories returned HTTP 200 with an empty categories array, making the nonexistent project look like a valid project without categories.
BUG-04: HEAD /projects/99999/categories returned HTTP 200 even though the project ID was invalid.
By comparison, GET /projects/99999 returned HTTP 404. The four cases were documented in individual bug summaries with reproduction steps and potential impact. The required behavior of these relationship endpoints still needs to be checked against Swagger before asserting that HTTP 404 is specified.

b) Supplementary Session: Projects API — XML (October 12, 2026)

After the JSON exploration, I performed a separate 11-minute XML verification. The purpose was to confirm whether the Projects API supported XML request bodies and XML responses for its main operations.
GET requests returned project information in XML, and POST /projects successfully created a project from an XML body with HTTP 201. POST and PUT updates reproduced the same distinction seen with JSON: POST preserved unspecified fields, while PUT reset the omitted description. XML project deletion was also successful, and a subsequent GET returned HTTP 404. A malformed XML request was rejected with HTTP 400, although its response Content-Type was text/html rather than application/xml.

The most important finding was that BUG-01 could also be reproduced in XML. GET /projects/99999/tasks returned HTTP 200 with tasks belonging to Project 1, despite requesting a nonexistent project. This is a reproduction of the same issue in another response format, rather than a fifth separate bug. Supporting terminal output was recorded during the session.

Structure of Unit Test Suite

1) Todos

The Todos tests are part of the same test suite as the Projects tests and use the same tools (Mocha, Chai, fetch() and fast-xml-parser), described in more detail below. In total, I created 61 regular API tests for the Todos section. These include 14 tests for the /todos collection, 13 for individual todos, eight for relationships between todos and projects, and seven for relationships between todos and categories. I also included six tests for undocumented API behavior, such as OPTIONS, PATCH, PUT and DELETE requests on the collection, as well as creating a project using its title. The remaining 13 tests cover XML requests.

For the six bugs identified, I followed the same approach as the Projects tests. Each bug has one test confirming the actual API behavior, which passes, and another test checking the expected behavior, which fails. The expected-behavior tests are kept separately in todoBugsExpected.js. The helper functions are in todoHelpers.js. Before running any test that modifies data, I save the current state of the todos, projects and categories. Once the test finishes, I compare the data with its original state and remove anything that was created. This helps make sure tests do not leave unwanted changes in the API.

For the test data, I decided to use tasks related to managing a soccer team. This gave me realistic examples to work with while keeping the automated tests different from the shopping list scenario used during exploratory testing.

Like the Projects tests, every Todos test file first checks that the API is running and fails with a clear message if it is not. The tests also cover filtering with query parameters, malformed JSON and XML payloads, and invalid operations such as updating or deleting a todo that does not exist.

2) Projects

The automated Projects tests use JavaScript with Mocha as the test runner and Chai for assertions. HTTP requests are made with the built-in fetch() function, while fast-xml-parser is used to validate XML responses. The test files are grouped by endpoint and separated into JSON and XML directories, following the general organization shown in the sample deliverable.

The Projects portion contains 43 regular API tests: 10 for the /projects collection, 17 for individual projects, nine for project–Todo relationships and seven for project–Category relationships. Four additional observed-behavior tests reproduce the reported issues. Four corresponding expected-behavior tests were placed in a separate module to expose the difference between the proposed HTTP 404 response and the implementation's HTTP 200 response.

The regular tests cover successful operations, nonexistent and deleted IDs, malformed JSON and XML, repeated deletion, filtering and changes to project relationships. They use temporary

projects and cleanup functions to avoid relying on fixed project IDs. The helpers also check that existing Todos and Categories are preserved after relationship tests.
The tests do not start or stop the Java service automatically. Instead, a readiness check fails if the API is unavailable. A separate script runs the passing tests in a pseudorandom order using a seed that can be reused to reproduce an execution sequence.

Source Code Repository

The Projects and Todos test files are organized in the shared ECSE429-PartA repository at https://github.com/olibreadstick/ECSE429-PartA. The main folders are test/JSON and test/XML, with shared testHelpers.js, testConstants.js and todoHelpers.js files under test/. The test/README.md contains the execution instructions, and scripts/runRandomizedTests.mjs is used for random execution. The capability scripts, session notes, bug summaries and test videos are also included in the repository.

```
test/
  JSON/   — Projects and Todos tests and bug-verification modules
  XML/    — XML Projects and Todos tests
  testHelpers.js
  testConstants.js
  todoHelpers.js
scripts/
  projectsCapabilities.sh
  todoCapabilities.sh
  runRandomizedTests.mjs
```

The passing tests, the expected-behavior bug tests and the randomized test runner can be executed separately from the project root:

```
npx mocha "test/{JSON,XML}/*.js" --ignore "test/JSON/projectBugsExpected.js" --ignore "test/JSON/todoBugsExpected.js"
npx mocha test/JSON/projectBugsExpected.js
npx mocha test/JSON/todoBugsExpected.js
node scripts/runRandomizedTests.mjs
```

Findings of Unit Test Suite Execution

1) Todos

The Todos tests confirmed the issues found during exploratory testing. All 61 regular Todos tests passed, along with the six tests checking the observed behavior of the bugs. When combined with the Projects tests, the full suite had 114 passing tests. The six expected-behavior tests for BUG-05 to BUG-10 failed as expected, with each failure matching the issue explained in its bug summary.

To check the tests didn't depend on each other, I ran the full 114-test suite in a few different random orders (including seed 42), and every run passed. The Todos tests that change data also compare the todos, projects and categories before and after, which helps catch cases where a test leaves data behind. Running the capability script first on the same API instance didn't change the results either.
The basic todo operations worked in both JSON and XML, but creating relationships through XML caused problems. The other issues the tests kept running into were the ones from the session: PUT wiping fields that weren't sent, the relationship endpoints answering for todos that don't exist, and todo–category links only showing up on one side. I also found that some rejected todo creation requests still used up an ID.

2) Projects
The automated tests generally confirmed the findings from the exploratory session. All 43 regular Projects tests passed. The four observed-behavior bug tests also passed, giving a total of 47 passing tests when the regular suite was run together. The four separate expected-behavior tests failed because they expected HTTP 404 for a deleted project's task or category relationship endpoint, while the implementation returned HTTP 200. These are reproducible discrepancies, but the proposed expected status still needs to be checked against the API documentation.
To investigate test independence, the 47-test passing suite was run in multiple pseudorandom orders, including seeds 42 and 123. Every randomized run produced 47 passing results. I also stopped the TodoManager service and confirmed that the tests failed their readiness check; after restarting the service, they passed again.
Finally, I compared the Projects, Todos and Categories collection responses before and after executing the suite. The before-and-after comparisons produced no differences. Together with the relationship-specific checks and cleanup functions, this provides evidence that the tests did not leave unexpected changes in those collections. A full comparison of every inverse relationship endpoint was not separately performed.
Overall, the normal Projects operations worked consistently across JSON and XML, while requests involving nonexistent parent projects showed unexpected behavior at the task and category relationship endpoints. The four observations are retained in the bug summaries pending a final comparison with the required Swagger responses.

Use of AI

Generative AI Usage Statement
ChatGPT was used as a supporting tool throughout the ECSE 429 project for the following purposes:

1. Setting up the coding environment

Prompt: "Please without overstepping, help me define or learn the general structure of a coding environment created with the intention of testing APIs should be set up using JavaScript, Mocha, and Chai."

2. Understanding the project capabilities script
Prompt: "Teach me how the project capabilities script should function in the context of this project and what its purpose is."

3. Clarifying potential bugs
Prompt: "I am having trouble discerning whether nonsequential todo IDs should be considered a bug. How would you perceive the situation?"

4. Understanding testing approaches
Prompt: "I need better context on why there is such a distinction between exploratory testing and automated testing. What are the main goals one should one be looking to accomplish in the context of each."

5. Understanding test independence
Prompt: "Why must automated tests be independent of one another, and how can running tests in random orders help verify this?"

6. Troubleshooting testing errors
Prompt: "What are main reasons for Mocha tests might fail when testing a REST API, and how do I identify whether the issue comes from the test or the application?"

ChatGPT was used to clarify concepts, better understand project requirements, and assist with troubleshooting. Testing decisions, identification of bugs, implementation, and verification of results remained my responsibility.